
FlexMatch: Boosting Semi-Supervised Learning with Curriculum Pseudo Labeling

Bowen Zhang*

Tokyo Institute of Technology
bowen.z.ab@m.titech.ac.jp

Yidong Wang*

Tokyo Institute of Technology
wang.y.ca@m.titech.ac.jp

Wenxin Hou

Microsoft
wenxinhou@microsoft.com

Hao Wu

Tokyo Institute of Technology
wu.h.aj@m.titech.ac.jp

Jindong Wang[†]

Microsoft Research Asia
jindwang@microsoft.com

Manabu Okumura[†]

Tokyo Institute of Technology
oku@pi.titech.ac.jp

Takahiro Shinozaki[†]

Tokyo Institute of Technology
shinot@ict.e.titech.ac.jp

Abstract

The recently proposed FixMatch achieved state-of-the-art results on most semi-supervised learning (SSL) benchmarks. However, like other modern SSL algorithms, FixMatch uses a pre-defined constant threshold for all classes to select unlabeled data that contribute to the training, thus failing to consider different learning status and learning difficulties of different classes. To address this issue, we propose Curriculum Pseudo Labeling (CPL), a curriculum learning approach to leverage unlabeled data according to the model's learning status. The core of CPL is to flexibly adjust thresholds for different classes at each time step to let pass informative unlabeled data and their pseudo labels. CPL does not introduce additional parameters or computations (forward or backward propagation). We apply CPL to FixMatch and call our improved algorithm *FlexMatch*. FlexMatch achieves state-of-the-art performance on a variety of SSL benchmarks, with especially strong performances when the labeled data are extremely limited or when the task is challenging. For example, FlexMatch achieves **13.96%** and **18.96%** error rate reduction over FixMatch on CIFAR-100 and STL-10 datasets respectively, when there are only 4 labels per class. CPL also significantly boosts the convergence speed, e.g., FlexMatch can use only 1/5 training time of FixMatch to achieve even better performance. Furthermore, we show that CPL can be easily adapted to other SSL algorithms and remarkably improve their performances. We open-source our code at <https://github.com/TorchSSL/TorchSSL>.

1 Introduction

Semi-supervised learning (SSL) has attracted increasing attention in recent years due to its superiority in leveraging a large amount of unlabeled data. This is particularly advantageous when the labeled data are limited in quantity or laborious to obtain. Consistency regularization [1–3] and pseudo labeling [4–8] are two powerful techniques for utilizing unlabeled data and have been widely used in modern SSL algorithms [9–13]. The recently proposed FixMatch [14] achieves competitive results

*Equal contribution.

[†]Corresponding author.

by combining these techniques with weak and strong data augmentations and using cross-entropy loss as the consistency regularization criterion.

However, a drawback of FixMatch and other popular SSL algorithms such as Pseudo-Labeling [4] and Unsupervised Data Augmentation (UDA) [11] is that they rely on a *fixed* threshold to compute the unsupervised loss, using only unlabeled data whose prediction confidence is above the threshold. While this strategy can make sure that only high-quality unlabeled data contribute to the model training, it ignores a considerable amount of other unlabeled data, especially at the early stage of the training process, where only a few unlabeled data have their prediction confidence above the threshold. Moreover, modern SSL algorithms handle all classes *equally* without considering their different learning difficulties.

To address these issues, we propose Curriculum Pseudo Labeling (CPL), a curriculum learning [15] strategy to take into account the learning status of each class for semi-supervised learning. CPL substitutes the pre-defined thresholds with *flexible* thresholds that are dynamically adjusted for each class according to the current learning status. Notably, this process does not introduce any additional parameter (hyperparameter or trainable parameter) or extra computation (forward or back propagation). We apply this curriculum learning strategy directly to FixMatch and call the improved algorithm *FlexMatch*.

While the training speed remains as efficient as that of FixMatch, FlexMatch converges significantly faster and achieves state-of-the-art performances on most SSL image classification benchmarks. The benefit of introducing CPL is particularly remarkable when the labels are scarce or when the task is challenging. For instance, on the STL-10 dataset, FlexMatch achieves relative performance improvement over FixMatch by 18.96%, 16.11%, and 7.68% when the label amount is 400, 2500, and 10000 respectively. Moreover, CPL further shows its superiority by boosting the convergence speed – with CPL, FlexMatch takes less than 1/5 training time of FixMatch to reach its final accuracy. Adapting CPL to other modern SSL algorithms also leads to improvements in accuracy and convergence speed.

To sum up, this paper makes the following three contributions:

- We propose Curriculum Pseudo Labeling (CPL), a curriculum learning approach of dynamically leveraging unlabeled data for SSL. It is almost cost-free and can be easily integrated to other SSL methods.
- CPL significantly boosts the accuracy and convergence performance of several popular SSL algorithms on common benchmarks. Specifically, FlexMatch, the integration of FixMatch and CPL, achieves state-of-the-art results.
- We open-source TorchSSL, a unified PyTorch-based semi-supervised learning codebase for the fair study of SSL algorithms. TorchSSL includes implementations of popular SSL algorithms and their corresponding training strategies, and is easy to use or customize.

2 Background

Consistency regularization follows the continuity assumption of SSL [1, 2]. The most basic consistency loss in SSL, such as in Π Model [9], Mean Teacher [10] and MixMatch [12], is the ℓ_2 loss:

$$\sum_{b=1}^{\mu B} \|p_m(y|\omega(u_b)) - p_m(y|\omega(u_b))\|_2^2, \quad (1)$$

where B is the batch size of labeled data, μ is the ratio of unlabeled data to labeled data, ω is a stochastic data augmentation function (thus the two terms in Eq.(1) are different), u_b denotes a piece of unlabeled data, and p_m represents the output probability of the model. With the introduction of pseudo labeling techniques [5, 7], the consistency regularization is converted to an entropy minimization process [16], which is more suitable for the classification task. The improved consistency loss with pseudo labeling can be represented as:

$$\frac{1}{\mu B} \sum_{b=1}^{\mu B} \mathbb{1}(\max(p_m(y|\omega(u_b))) > \tau) H(\hat{p}_m(y|\omega(u_b)), p_m(y|\omega(u_b))), \quad (2)$$

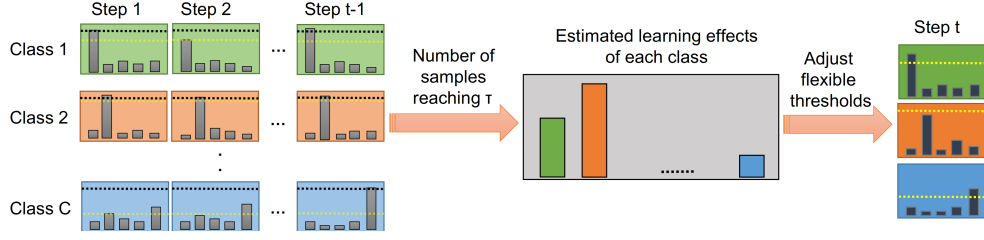


Figure 1: Illustration of Curriculum Pseudo Label (CPL). The estimated learning effects of each class are decided by the number of unlabeled data samples falling into this class and above the fixed threshold. They are then used to adjust the flexible thresholds to let pass the optimal unlabeled data. Note that the estimated learning effects do not always grow – they may also decrease if the predictions of the unlabeled data fall into other classes in later iterations.

where H is cross-entropy, τ is the pre-defined threshold and $\hat{p}_m(y|\omega(u_b))$ is the pseudo label that can either be a ‘hard’ one-hot label [4, 14] or a sharpened ‘soft’ one [11]. The intention of using a threshold is to mask out noisy unlabeled data that have low prediction confidence.

FixMatch utilizes such consistency regularization with strong augmentation to achieve competitive performance. For unlabeled data, FixMatch first uses weak augmentation to generate artificial labels. These labels are then used as the target of strongly-augmented data. The unsupervised loss term in FixMatch thereby has the form:

$$\frac{1}{\mu B} \sum_{b=1}^{\mu B} \mathbb{1}(\max(p_m(y|\omega(u_b))) > \tau) H(\hat{p}_m(y|\omega(u_b)), p_m(y|\Omega(u_b))), \quad (3)$$

where Ω is a strong augmentation function instead of weak augmentation ω .

Of the aforementioned works, the pre-defined threshold (τ) is constant. We believe this can be improved because the data of some classes may be inherently more difficult to learn than others. Curriculum learning [15] is a learning strategy where learning samples are gradually introduced according to the model’s learning process. In such a way, the model is always optimally challenged. This technique is widely employed in deep learning research [17–21].

3 FlexMatch

3.1 Curriculum Pseudo Labeling

While current SSL algorithms render pseudo labels of only high-confidence unlabeled data cut off by a pre-defined threshold, CPL renders the pseudo labels *to different classes* and *at different time steps*. Such a process is realized by adjusting the thresholds according to the model’s learning status of each class.

However, it is non-trivial to dynamically determine the thresholds according to the learning status. The most ideal approach would be calculating evaluation accuracies for each class and use them to scale the threshold, as:

$$\mathcal{T}_t(c) = a_t(c) \cdot \tau, \quad (4)$$

where $\mathcal{T}_t(c)$ is the flexible threshold for class c at time step t and $a_t(c)$ is the corresponding evaluation accuracy. In this way, lower accuracy that indicates a less satisfactory learning status of the class will lead to a lower threshold that encourages more samples of this class to be learned. Since we cannot use the evaluation set in the model learning process, one may have to separate an extra validation set from the training set for such accuracy evaluations. However, this practice show two fatal problems: First, such a *labeled* validation set separated from the training set is expensive under SSL scenario as the labeled data are already scarce. Second, to dynamically adjust the thresholds in the training process, accuracy evaluations must be done continually at each time step t , which will considerably slow down the training speed.

In this work, we propose Curriculum Pseudo Labeling (CPL) for semi-supervised learning. Our CPL uses an alternative way to estimate the learning status, which does not introduce additional inference processes, nor needs an extra validation set. As believed in [14], a high threshold that filters out noisy pseudo labels and leaves only high-quality ones can considerably reduce the confirmation bias [22]. Therefore, our key assumption is that when the threshold is high, the learning effect of a class can be reflected by the number of samples whose predictions fall into this class and above the threshold. Namely, the class with fewer samples having their prediction confidence reach the threshold is considered to have a greater learning difficulty or a worse learning status, formulated as:

$$\sigma_t(c) = \sum_{n=1}^N \mathbb{1}(\max(p_{m,t}(y|u_n)) > \tau) \cdot \mathbb{1}(\arg \max(p_{m,t}(y|u_n) = c). \quad (5)$$

where $\sigma_t(c)$ reflects the learning effect of class c at time step t . $p_{m,t}(y|u_n)$ is the model's prediction for unlabeled data u_n at time step t , and N is the total number of unlabeled data. When the unlabeled dataset is balanced (i.e., the number of unlabeled data belonging to different classes are equal or close), larger $\sigma_t(c)$ indicates a better estimated learning effect. By applying the following normalization to $\sigma_t(c)$ to make its range between 0 to 1, it can then be used to scale the fixed threshold τ :

$$\beta_t(c) = \frac{\sigma_t(c)}{\max_c \sigma_t}, \quad (6)$$

$$\mathcal{T}_t(c) = \beta_t(c) \cdot \tau. \quad (7)$$

One characteristic of such a normalization approach is that the best-learned class has its $\beta_t(c)$ equal to 1, causing its flexible threshold equal to τ . This is desirable. For classes that are hard to learn, the thresholds are lowered down, encouraging more training samples in these classes to be learned. This also improves the data utilization ratio. As learning proceeds, the threshold of a well-learned class is raised higher to selectively pick up higher-quality samples. Eventually, when all classes have reached reliable accuracies, the thresholds will all approach τ . Note that the thresholds do not always grow, it may also decrease if the unlabeled data is classified into a different class in later iterations. This new threshold is used for calculating the unsupervised loss in FlexMatch, which can be formulated as:

$$\mathcal{L}_{u,t} = \frac{1}{\mu B} \sum_{b=1}^{\mu B} \mathbb{1}(\max(q_b) > \mathcal{T}_t(\arg \max(q_b))) H(\hat{q}_b, p_m(y|\Omega(u_b))), \quad (8)$$

where $q_b = p_m(y|\omega(u_b))$. The flexible thresholds are updated at each iteration. Finally, we can formulate the loss in FlexMatch as the weighted combination (by λ) of supervised and unsupervised loss:

$$\mathcal{L}_t = \mathcal{L}_s + \lambda \mathcal{L}_{u,t}, \quad (9)$$

where $\mathcal{L}_s$ is the supervised loss on labeled data:

$$\mathcal{L}_s = \frac{1}{B} \sum_{b=1}^B H(y_b, p_m(y|\omega(x_b))). \quad (10)$$

Note that the cost of introducing CPL is almost free. Practically, every time the prediction confidence of an unlabeled data u_n is above the fixed threshold τ , the data, and its predicted class are marked and will be used for calculating $\beta_t(c)$ at the next time step. Such marking actions are bonus actions each time the consistency loss is computed. Therefore, FlexMatch does not introduce additional forward propagation processes for evaluating the model's learning status, nor new parameters.

3.2 Threshold warm-up

We noticed in our experiments that at the early stage of the training, the model may blindly predict most unlabeled samples into a certain class depending on the parameter initialization (i.e., more likely to have confirmation bias). Hence, the estimated learning status may not be reliable at this stage. Therefore, we introduce a warm-up process by rewriting the denominator in Eq. (6) as:

$$\beta_t(c) = \frac{\sigma_t(c)}{\max \left\{ \max_c \sigma_t, N - \sum_c \sigma_t \right\}}, \quad (11)$$

Algorithm 1 FlexMatch algorithm.

```
1: Input:  $\mathcal{X} = \{(x_m, y_m) : m \in (1, \dots, M)\}$ ,  $\mathcal{U} = \{u_n : n \in (1, \dots, N)\}$  {M labeled data and N unlabeled data.}
2:  $\hat{u}_n = -1 : n \in (1, \dots, N)$  {Initialize predictions of all unlabeled data as -1 indicating unused.}
3: while not reach the maximum iteration do
4:   for  $c = 1$  to  $C$  do
5:      $\sigma(c) = \sum_{n=1}^N \mathbb{1}(\hat{u}_n = c)$  {Compute estimated learning effect.}
6:     if  $\max \sigma(c) < \sum_{n=1}^N \mathbb{1}(\hat{u}_n = -1)$  then
7:       Calculate  $\beta(c)$  using Eq. (11) {Threshold warms up when unused data dominate.}
8:     else
9:       Calculate  $\beta(c)$  using Eq. (6) {Compute normalized estimated learning effect.}
10:    end if
11:    Calculate  $\mathcal{T}(c)$  using Eq. (7) {Determine the flexible threshold for class  $c$ .}
12:  end for
13:  for  $b = 1$  to  $\mu B$  do
14:    if  $p_m(y|\omega(u_b)) > \tau$  then
15:       $\hat{u}_b = \arg \max q_b$  {Update the prediction of unlabeled data  $u_b$ .}
16:    end if
17:  end for
18:  Compute the loss via Eq. (8), (10) and (9).
19: end while
20: Return: Model parameters.
```

where the term $N - \sum_{c=1}^C \sigma_t(c)$ can be regarded as the number of unlabeled data that have not been used. This ensures that at the beginning of the training, all estimated learning effects gradually rise from 0 until the number of unused unlabeled data is no longer predominant. The duration of such a period depends on the unlabeled data amount (ref. N in Eq. (11)) and the learning difficulty (ref. the growing speed of $\sigma_t(c)$ in Eq. (11)) of the dataset. In practice, such a warm-up process is very easy to implement as we can add an extra class to denote the unused unlabeled data. Thus calculating the denominator of Eq. (11) is simply converted to finding the maximum among $c + 1$ classes.

3.3 Non-linear mapping function

The flexible threshold in Eq. (7) is determined by the normalized estimated learning effects via a linear mapping. However, it may not be the most suitable mapping in the real training process, where the increase or decrease of $\beta_t(c)$ may make big jumps in the early phase where the predictions of the model are still unstable; and only make small fluctuations after the class is well-learned in the mid and late training stage. Therefore, it is preferable if the flexible thresholds can be more sensitive when $\beta_t(c)$ is large and vice versa.

We propose a non-linear mapping function to enable the thresholds to have a non-linear increasing curve when $\beta_t(c)$ ranges uniformly from 0 to 1, as formulated below:

$$\mathcal{T}_t(c) = \mathcal{M}(\beta_t(c)) \cdot \tau, \quad (12)$$

where $\mathcal{M}(\cdot)$ is a non-linear mapping function. It is clear that Eq. (7) can be seen as a special case by setting $\mathcal{M}$ to the identity function. The mapping function $\mathcal{M}$ should be monotonically increasing and have a maximum no larger than $1/\tau$ (otherwise the flexible threshold can be larger than 1 and filter out all samples). To avoid introducing additional hyperparameters (e.g. lower limits of the flexible thresholds), we consider the mapping function to have a range from 0 to 1 so that the flexible thresholds range from 0 to τ .

A monotone increasing convex function lets the thresholds grow slowly when $\beta_t(c)$ is small, and become more sensitive as $\beta_t(c)$ gets larger. Hence, we intuitively choose a convex function with the above-mentioned properties $\mathcal{M}(x) = \frac{x}{2-x}$ for our experiments. We also conduct an ablation study to compare among mapping functions with different convexity and concavity in Sec. 4.4. The full algorithm of FlexMatch is shown in Algorithm 1.

Table 1: Error rates on CIFAR-10/100, SVHN, and STL-10 datasets. The ‘Flex’ prefix denotes applying CPL to the algorithm, and ‘PL’ is an abbreviation of Pseudo-Labeling. STL-10 dataset does not have label information for unlabeled data, thus its fully-supervised result is unavailable.

Dataset	CIFAR-10			CIFAR-100			STL-10			SVHN	
Label Amount	40	250	4000	400	2500	10000	40	250	1000	40	1000
PL	74.61 $\pm$ 0.26	46.49 $\pm$ 2.20	15.08 $\pm$ 0.19	87.45 $\pm$ 0.85	57.74 $\pm$ 0.28	36.55 $\pm$ 0.24	74.68 $\pm$ 0.99	55.45 $\pm$ 2.43	32.64 $\pm$ 0.71	64.61 $\pm$ 5.60	9.40 $\pm$ 0.32
Flex-PL	73.74 $\pm$ 1.96	46.14 $\pm$ 1.81	14.75 $\pm$ 0.19	85.72 $\pm$ 0.46	56.12 $\pm$ 0.51	35.60 $\pm$ 0.15	73.42 $\pm$ 2.19	52.06 $\pm$ 2.50	32.05 $\pm$ 0.37	63.21 $\pm$ 3.64	12.05 $\pm$ 0.54
UDA	10.62 $\pm$ 3.75	5.16 $\pm$ 0.06	4.29 $\pm$ 0.07	46.39 $\pm$ 1.59	27.73 $\pm$ 0.21	22.49 $\pm$ 0.23	37.42 $\pm$ 8.44	9.72 $\pm$ 1.15	6.64 $\pm$ 0.17	5.12 $\pm$ 4.27	1.89 $\pm$ 0.01
Flex-UDA	5.44 $\pm$ 0.52	5.02 $\pm$ 0.07	4.24 $\pm$ 0.06	45.17 $\pm$ 1.88	27.08 $\pm$ 0.15	21.91 $\pm$ 0.10	29.53 $\pm$ 2.10	9.03 $\pm$ 0.45	6.10 $\pm$ 0.25	3.42 $\pm$ 1.51	2.02 $\pm$ 0.05
FixMatch	7.47 $\pm$ 0.28	4.86 $\pm$ 0.05	4.21 $\pm$ 0.08	46.42 $\pm$ 0.82	28.03 $\pm$ 0.16	22.20 $\pm$ 0.12	35.97 $\pm$ 4.14	9.81 $\pm$ 1.04	6.25 $\pm$ 0.33	3.81 $\pm$ 1.18	1.96 $\pm$ 0.03
FlexMatch	4.97 $\pm$ 0.06	4.98 $\pm$ 0.09	4.19 $\pm$ 0.01	39.94 $\pm$ 1.62	26.49 $\pm$ 0.20	21.90 $\pm$ 0.15	29.15 $\pm$ 4.16	8.23 $\pm$ 0.39	5.77 $\pm$ 0.18	8.19 $\pm$ 3.20	6.72 $\pm$ 0.30
Fully-Supervised	4.62 $\pm$ 0.05			19.30 $\pm$ 0.09			-			2.13 $\pm$ 0.02	

4 Experiments

We evaluate FlexMatch and other CPL-enabled algorithms on common SSL datasets: CIFAR-10/100 [23], SVHN [24], STL-10 [25] and ImageNet [26], and extensively investigate the performance under various labeled data amounts. We mainly compare our method with Pseudo-Labeling [4], UDA [11] and FixMatch [14], since they all involve a pre-defined threshold. The results of other popular SSL algorithms are in the appendix B. We also add a fully-supervised experiment for each dataset to better understand the results of SSL algorithms. Note that previously suggested [27] fully-supervised comparisons use only the labeled set for training, whose purpose is to manifest the improvement brought by the introduction of unlabeled data. With the development of modern SSL algorithms, however, semi-supervised approaches are achieving competitive performance with supervised ones, or even better performance due to the strength of consistency regularization. Therefore, our fully-supervised comparisons are conducted with all data labeled, and apply weak data augmentations following Eq. (10). We re-implement all baselines using our PyTorch [28] codebase: TorchSSL, which is introduced in the appendix B.

For a fair comparison, we use the same hyperparameters following FixMatch [14]. Concretely, the optimizer for all experiments is standard stochastic gradient descent (SGD) with a momentum of 0.9 [29, 30]. For all datasets, we use an initial learning rate of 0.03 with a cosine learning rate decay schedule [31] as $\eta = \eta_0 \cos(\frac{7\pi k}{16K})$, where η_0 is the initial learning rate, k is the current training step and K is the total training step that is set to 2^{20} . We also perform an exponential moving average with the momentum of 0.999. The batch size of labeled data is 64 except for ImageNet. μ is set to be 1 for Pseudo-Label and 7 for UDA, FixMatch, and FlexMatch. τ is set to 0.8 for UDA and 0.95 for Pseudo Label, FixMatch, and FlexMatch. These setups follow the original papers. The strong augmentation function used in our experiments is RandAugment [32]. We use ResNet-50 [33] for the ImageNet experiment and Wide ResNet (WRN) [34] and its variant [35] for other datasets. Detailed hyperparameters are listed in the appendix A.

We adopt two evaluation metrics: (1) the median error rate of the last 20 checkpoints following [12, 14], and (2) the best error rate in all checkpoints. We argue that the median approach is not suitable when the convergence speeds of the algorithms show significant differences – the large number of redundant iterations may result in over-fitting for the fast-converge algorithms. Therefore, we report the best error rates for all algorithms, while the results of the median approach are also provided in the appendix A, showing that our FlexMatch still achieves the best performance. We run each task three times using distinct random seeds to obtain the error bars.

4.1 Main results

The classification error rates on CIFAR-10/100, STL-10 and SVHN datasets are in Table 1, and the results on ImageNet are in Sec. 4.2. Note that the SVHN dataset used in our experiment also includes the extra set that contains 531,131 additional samples. Results demonstrate that FlexMatch achieves the state-of-the-art performance on most of the benchmark datasets except for SVHN where Flex-UDA (i.e., UDA with CPL) and UDA have the lowest error rate on the 40-label split and the 1000-label split, respectively. We also provide the detailed precision, recall, F1, and AUC results in the appendix A. Our CPL (FlexMatch) has the following advantages:

CPL achieves better performance on tasks with extremely limited labeled data. Our FlexMatch significantly outperforms other methods when the amount of labels is extremely small. For

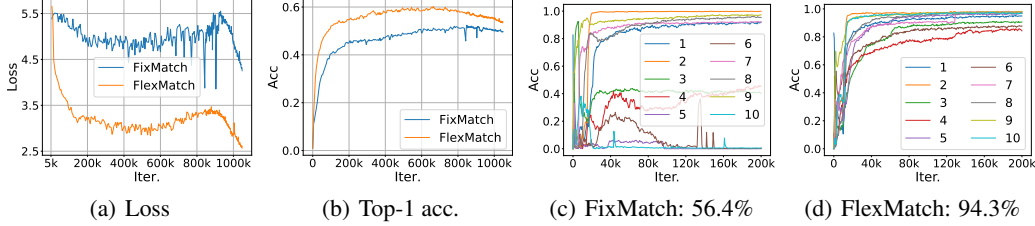


Figure 3: Convergence analysis of FixMatch and FlexMatch. (a) and (b) depict the loss and top-1-accuracy on CIFAR-100 with 400 labels. Evaluations are done every 5K iterations. (c) and (d) demonstrate the class-wise accuracy within the first 200K iterations on CIFAR-10 dataset. The numbers in legend correspond to the ten classes in the dataset.

instance, on the CIFAR-100 dataset with 400 labels (i.e., only 4 label samples per class), FlexMatch achieves an average error rate of **39.94%**, which significantly outperforms FixMatch (46.42%).

CPL improves the performance of existing SSL algorithms.

Other than FixMatch, CPL can also improve the performance of other existing SSL algorithms such as Pseudo-Labeling and UDA. For instance, the error rate is reduced from 37.4% to 29.53% for UDA on the STL-10 40-label split after introducing CPL (refer to as Flex-UDA in Table 1). These results further prove the effectiveness of CPL in better leveraging unlabeled data. Figure 2 shows the average running time of a single iteration with or without adding our CPL, it is clear that while improving the performance of existing SSL algorithms, our CPL *does not* introduce additional computational burden.

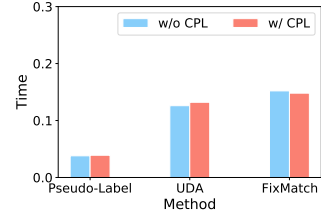


Figure 2: Average running time of one iteration on a single GeForce RTX 3090 GPU.

CPL achieves better performance on complicated tasks.

The STL-10 dataset contains unlabeled data from a similar but broader distribution of images than its labeled set. The existence of new types of objects in the unlabeled dataset makes STL-10 a more challenging and realistic task. FlexMatch achieves greater performance improvement under such a challenging situation. The error rate on STL-10 with only 40 labels is 29.15%, which is relatively **18.96%** better than FixMatch (35.97%). Similar strong improvements are also observed on CIFAR-100 dataset, which has as many as one hundred classes.

We also analyze the reason why FlexMatch performs less favorably on SVHN. This is probably because SVHN is a relatively *simple* (i.e., to classify digits) yet *unbalanced* dataset. The class-wise imbalance leads to the classes with fewer samples never have their estimated learning effects close to 1 according to Eq. (6), even when they are already well-learned. Such low thresholds allow noisy pseudo-labeled samples to be trusted and learned throughout the training process, which is also reflected by the loss descent curve where the low-threshold classes have major fluctuations. FixMatch, on the other hand, fixes its threshold at 0.95 to filter out noisy samples. Such a fixed high threshold is not preferable with respect to both accuracies of hard-to-learn classes and overall convergence speed as explained earlier, but since SVHN is an easy task, the model can easily learn the task and make high-confidence predictions, setting a high-fixed threshold thus becomes less problematic and has its advantages outweighed.

4.2 Results on ImageNet

We also verify the effectiveness of CPL on ImageNet-1K [26] which is a much more realistic and complicated dataset. We randomly choose the same 100K labeled data (i.e., 100 labels per class), which is less than 8% of the total labels. The hyper-parameters used for ImageNet can be found in the appendix A, where the two algorithms share the same hyper-parameters. We show the error rate comparison after running 2^{20} iterations in Table 2. This result indicates that when the task is complicated, despite the class imbalance issue (the number

Table 2: Error rate results on ImageNet after 2^{20} iterations.

Method	Top-1	Top-5
FixMatch	43.66	21.80
FlexMatch	41.85	19.48

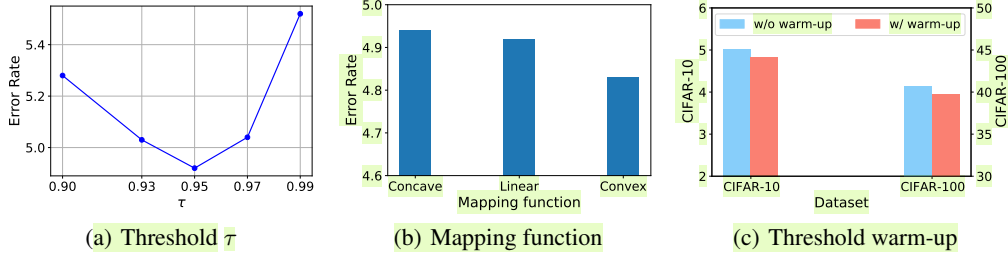


Figure 4: Ablation study of FlexMatch.

of images within each class ranges from 732 to 1300), CPL can still bring improvements. Note that this result does not represent the best performance of each algorithm as the model cannot fully converge after 2^{20} iterations, and due to the computational resource limitation, we did not further tune the hyper-parameters to obtain the best results on ImageNet.

4.3 Convergence speed acceleration

Another strong advantage of FlexMatch is its superior convergence speed. Figure 3(a) and 3(b) shows the comparison between FlexMatch and FixMatch with respect to the loss and top-1-accuracy on CIFAR-100 400-label split. The loss of FlexMatch decreases much faster and smoother than FixMatch, demonstrating its superior convergence speed. The major fluctuations of the loss in FixMatch may due to the pre-defined threshold that lets pass most unlabeled data belonging to certain classes, whereas with CPL a larger batch of unlabeled data containing samples from various classes enables the gradient to more directly head toward the global optimum. As a result, with only 50K iterations, FlexMatch has already surpassed the final results of FixMatch. After 800K iterations, however, we observe a further decrease in loss and accuracy. This is likely due to over-fitting, which also occurs in FixMatch after 900K iterations. Thus, we believe it is not fair to use the median results of the last few checkpoints for evaluating algorithms with different convergence speeds.

We further compare the class-wise accuracy of FixMatch and FlexMatch on CIFAR-10 in their early training stages. As shown in Figure 3(c) and 3(d), at iteration 200K, FixMatch only hits an overall accuracy of 56.35% as half of the classes are still learned unsatisfactorily, whereas FlexMatch has already achieved an overall accuracy of 94.29% which is even higher than the final accuracy reached by FixMatch after 1M iterations. It is manifest that the introduction of CPL successfully encourages the model to proactively learn those difficult classes thereby improving the overall learning effect.

4.4 Ablation study

We conduct experiments to evaluate three components of FlexMatch: the upper limit of thresholds τ , mapping functions $\mathcal{M}(x)$, and threshold warm-up.

Threshold upper bound. We investigate 5 different τ values and 3 different mapping functions on CIFAR-10 dataset with 40 labels. As shown in Figure 4(a), the optimal choice of τ is around 0.95, either increasing or decreasing this value results in a performance decay. Note that in FlexMatch, tuning τ does not only affect the upper limit of the threshold but also the estimated learning effects because they are determined by the number of samples that fall above τ .

Mapping function. We explore three different mapping functions in Figure 4(b): (1) concave: $\mathcal{M}(x) = \ln(x + 1) / \ln 2$, (2) linear: $\mathcal{M}(x) = x$, and (3) convex: $\mathcal{M}(x) = x / (2 - x)$. We see that the convex function shows the best performance and the concave function shows the worst. Although tweaking the degree of convexity may probably lead to further improvement, we do not make further investigation in this paper. It is noteworthy that all these functions have their outputs grow from 0 to 1 when the inputs go from 0 to 1. One may also design a function with a different range, for instance, from 0.5 to 1. In this case, it is equivalent to setting a lower limit to the flexible threshold so that even at the beginning of the training, only samples with prediction confidence higher than this limit will contribute to the unsupervised loss. We do not include such a lower limit in FlexMatch since it will introduce a new hyperparameter. However, we did find that setting a lower limit at 0.5 can slightly

improve the performance. A possible reason is that the lower threshold prevents noisy training caused by incorrect pseudo labels at the early stage [36].

Threshold warm-up. We analyze the performance of threshold warm-up on both CIFAR-10 (40 labels) and CIFAR-100 (400 labels) datasets. As shown in Figure 4(c), threshold warm-up can bring about 0.2% absolute improvement on CIFAR-10 and about 1% on CIFAR-100. At the beginning of the training without the threshold warm-up, the flexible thresholds may go through heavy fluctuations because the denominator in Eq.(6) is small. In the meantime, there will always be some classes whose flexible thresholds reach or approach τ , thereby filtering out most unlabeled data in the batch. The threshold warm-up solves this issue by gradually raising the thresholds of all classes from zero – it creates a learning boom at the early training stage where most of the unlabeled data can be utilized.

Comparison with class balancing objectives. CPL has the effect of balancing across classes the number of unlabeled samples used to compute pseudo-labeling loss in each batch. Similar effect can be achieved by making the marginal class distribution close to a uniform distribution for each batch. We conduct such a comparative experiment by directly adding an additional objective to FixMatch: $\mathcal{L}_b = \sum_c q_c \log(q_c/\hat{p}_c)$ [22], where $\hat{p}_c$ is the mean predicted probability of class c across all samples in the batch, and q is a uniform distribution: $q_c = 1/C$. The error rate of adding such an objective is 7.16% on the CIFAR-10 40-label split (compared with FixMatch $7.47\% \pm 0.28$ and FlexMatch $4.97\% \pm 0.06$). While this approach requires instances of each class within each batch to be balanced to make sense, CPL does not have such a constraint. It is more flexible and involves less human intervention to adjust thresholds than adjusting model’s predictions.

5 Related Work

Pseudo-Labeling [4] is a pioneer SSL method that uses hard artificial labels converted from model predictions. A confidence-based strategy was used in [6] along with pseudo labeling so that the unlabeled data are used only when the predictions are sufficiently confident. Such confidence-based thresholding also presents in recently proposed UDA [11] and FixMatch [14] with the difference being that UDA used sharpened ‘soft’ pseudo labels with a temperature whereas Fixmatch adopted one-hot ‘hard’ labels. The success of UDA and FixMatch, however, relies heavily on the usage of strong data augmentations to improve the consistency regularization. ReMixMatch [13] also leveraged such strong augmentations.

The combination of curriculum learning and semi-supervised learning is popular in recent years [37–39]. For multi-model image classification task, [37] optimized the learning process of unlabeled images by judging their reliability and discriminability. In [38], the easy image-level properties are learned first and then used to facilitate segmentation via constrained CNNs. Curriculum learning is also used to alleviate out-of-distribution problems by picking up in-distribution samples from unlabeled data according to the out-of-distribution scores [39].

Several researches have investigated on dynamic threshold in related fields such as sentiment analysis [40] and semantic segmentation [41]. In [40], the threshold was gradually reduced to make high-quality data selected into labeled data set in the early stage and large-quantity in the later stage. An extra classifier is added to automate the threshold to deal with domain inconsistency in [41]. [42] introduced curriculum learning to self-training with a steadily increasing threshold and achieved near state-of-the-art results.

6 Conclusion and Future Work

In this paper, we introduce Curriculum Pseudo Labeling (CPL), a curriculum learning approach of leveraging unlabeled data for SSL. CPL dramatically improves the performance and convergence speed of SSL algorithms that involve thresholds while being extremely simple and almost cost-free. FlexMatch, our improved algorithm of FixMatch, achieves state-of-the-art performance on a variety of SSL benchmarks. In future work, we would like to improve our method under the long-tail scenario where the unlabeled data belonging to each class are extremely unbalanced.

Broader Impact

CPL fills the gap that no modern SSL algorithm considers the inherent learning difficulties of different classes during the training, and shows that by doing so, the convergence speed and final accuracy can both be improved. We hope that CPL can attract more future attention to explore the effectiveness of utilizing unlabeled data according to the model's learning status as well as the per-class learning difficulty.

Funding Disclosure

Funding in direct support of this work: computing resource granted by Tokyo Institute of Technology and Microsoft Research Asia. This work was partially supported by Toray Science Foundation.

References

- [1] Philip Bachman, Ouais Alsharif, and Doina Precup. Learning with pseudo ensembles. In *NeurIPS*, pages 3365–3373, 2014.
- [2] Samuli Laine and Timo Aila. Temporal ensembling for semi-supervised learning. *ICLR*, 2017.
- [3] Mehdi Sajjadi, Mehran Javanmardi, and Tolga Tasdizen. Regularization with stochastic transformations and perturbations for deep semi-supervised learning. In *NeurIPS*, pages 1171–1179, 2016.
- [4] Dong Hyun Lee et al. Pseudo label: The simple and efficient semi-supervised learning method for deep neural networks. In *Workshop on challenges in representation learning, ICML*, volume 3, 2013.
- [5] Geoffrey J McLachlan. Iterative reclassification procedure for constructing an asymptotically optimal rule of allocation in discriminant analysis. *Journal of the American Statistical Association*, 70(350):365–369, 1975.
- [6] Chuck Rosenberg, Martial Hebert, and Henry Schneiderman. Semi-supervised self-training of object detection models. 2005.
- [7] Henry Scudder. Probability of error of some adaptive pattern recognition machines. *IEEE Transactions on Information Theory*, 11(3):363–371, 1965.
- [8] Qizhe Xie, Minh Thang Luong, Eduard Hovy, and Quoc V Le. Self-training with noisy student improves imagenet classification. In *CVPR*, pages 10687–10698, 2020.
- [9] Antti Rasmus, Harri Valpola, Mikko Honkala, Mathias Berglund, and Tapani Raiko. Semi-supervised learning with ladder networks. In *NeurIPS*, pages 3546–3554, 2015.
- [10] Antti Tarvainen and Harri Valpola. Mean teachers are better role models: Weight averaged consistency targets improve semi-supervised deep learning results. In *NeurIPS*, pages 1195–1204, 2017.
- [11] Qizhe Xie, Zihang Dai, Eduard Hovy, Thang Luong, and Quoc Le. Unsupervised data augmentation for consistency training. *NeurIPS*, 33, 2020.
- [12] David Berthelot, Nicholas Carlini, Ian Goodfellow, Nicolas Papernot, Avital Oliver, and Colin Raffel. Mixmatch: A holistic approach to semi-supervised learning. *NeurIPS*, page 5050–5060, 2019.
- [13] David Berthelot, Nicholas Carlini, Ekin D Cubuk, Alex Kurakin, Kihyuk Sohn, Han Zhang, and Colin Raffel. Remixmatch: Semi-supervised learning with distribution matching and augmentation anchoring. In *ICLR*, 2019.
- [14] Kihyuk Sohn, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin A Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun-Liang Li. Fixmatch: Simplifying semi-supervised learning with consistency and confidence. *NeurIPS*, 33, 2020.

- [15] Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *ICML*, pages 41–48, 2009.
- [16] Yves Grandvalet, Yoshua Bengio, et al. Semi-supervised learning by entropy minimization. In *CAP*, pages 281–296, 2005.
- [17] Anastasia Pentina, Viktoriia Sharmanska, and Christoph H Lampert. Curriculum learning of multiple tasks. In *CVPR*, pages 5492–5500, 2015.
- [18] Lu Jiang, Deyu Meng, Qian Zhao, Shiguang Shan, and Alexander Hauptmann. Self-paced curriculum learning. In *AAAI*, volume 29, 2015.
- [19] Alex Graves, Marc G Bellemare, Jacob Menick, Remi Munos, and Koray Kavukcuoglu. Automated curriculum learning for neural networks. In *ICML*, pages 1311–1320. PMLR, 2017.
- [20] Guy Hacohen and Daphna Weinshall. On the power of curriculum learning in training deep networks. In *ICML*, pages 2535–2544. PMLR, 2019.
- [21] Daphna Weinshall, Gad Cohen, and Dan Amir. Curriculum learning by transfer learning: Theory and experiments with deep networks. In *ICML*, pages 5238–5246. PMLR, 2018.
- [22] Eric Arazo, Diego Ortego, Paul Albert, Noel E O’Connor, and Kevin McGuinness. Pseudo-labeling and confirmation bias in deep semi-supervised learning. In *IJCNN*, pages 1–8, 2020.
- [23] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.
- [24] Yuval Netzer, Tao Wang, Adam Coates, Alessandro Bissacco, Bo Wu, and Andrew Y Ng. Reading digits in natural images with unsupervised feature learning. 2011.
- [25] Adam Coates, Andrew Ng, and Honglak Lee. An analysis of single-layer networks in unsupervised feature learning. In *AISTATS*, pages 215–223, 2011.
- [26] Jia Deng, Wei Dong, Richard Socher, Li Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *CVPR*, pages 248–255. Ieee, 2009.
- [27] Avital Oliver, Augustus Odena, Colin Raffel, Ekin D Cubuk, and Ian J Goodfellow. Realistic evaluation of deep semi-supervised learning algorithms. In *NeurIPS*, pages 3239–3250, 2018.
- [28] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *NeurIPS*, 32:8026–8037, 2019.
- [29] Ilya Sutskever, James Martens, George Dahl, and Geoffrey Hinton. On the importance of initialization and momentum in deep learning. In *ICML*, pages 1139–1147. PMLR, 2013.
- [30] Boris T Polyak. Some methods of speeding up the convergence of iteration methods. *Ussr computational mathematics and mathematical physics*, 4(5):1–17, 1964.
- [31] Ilya Loshchilov and Frank Hutter. Sgdr: Stochastic gradient descent with warm restarts. *ICLR*, 2016.
- [32] Ekin D Cubuk, Barret Zoph, Jonathon Shlens, and Quoc V Le. Randaugment: Practical automated data augmentation with a reduced search space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 702–703, 2020.
- [33] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- [34] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. In *BMVC*, 2016.
- [35] Tianyi Zhou, Shengjie Wang, and Jeff Bilmes. Time-consistent self-supervision for semi-supervised learning. In *ICML*, pages 11523–11533. PMLR, 2020.

- [36] Mamshad Nayeem Rizve, Kevin Duarte, Yogesh S Rawat, and Mubarak Shah. In defense of pseudo-labeling: An uncertainty-aware pseudo-label selection framework for semi-supervised learning. *arXiv preprint arXiv:2101.06329*, 2021.
- [37] Chen Gong, Dacheng Tao, Stephen J Maybank, Wei Liu, Guoliang Kang, and Jie Yang. Multi-modal curriculum learning for semi-supervised image classification. *IEEE Transactions on Image Processing*, 25(7):3249–3260, 2016.
- [38] Hoel Kervadec, Jose Dolz, Éric Granger, and Ismail Ben Ayed. Curriculum semi-supervised segmentation. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pages 568–576. Springer, 2019.
- [39] Qing Yu, Daiki Ikami, Go Irie, and Kiyoharu Aizawa. Multi-task curriculum framework for open-set semi-supervised learning. In *ECCV*, pages 438–454. Springer, 2020.
- [40] Yue Han, Yuhong Liu, and Zhigang Jin. Sentiment analysis via semi-supervised learning: a model based on dynamic threshold and multi-classifiers. *Neural Computing and Applications*, 32(9):5117–5129, 2020.
- [41] Zhedong Zheng and Yi Yang. Rectifying pseudo-label learning via uncertainty estimation for domain-adaptive semantic segmentation. *IJCV*, 129(4):1106–1120, 2021.
- [42] Paola Cascante Bonilla, Fuwen Tan, Yanjun Qi, and Vicente Ordonez. Curriculum labeling: Revisiting pseudo-labeling for semi-supervised learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6912–6920, 2021.
- [43] Kihyuk Sohn, David Berthelot, Nicholas Carlini, Zizhao Zhang, Han Zhang, Colin A Raffel, Ekin Dogus Cubuk, Alexey Kurakin, and Chun Liang Li. fixmatch. <https://github.com/google-research/fixmatch>, 2020.
- [44] Lee Doyup and Cheon Yeongjae. Fixmatch-pytorch. <https://github.com/LeeDoYup/FixMatch-pytorch>, 2020.
- [45] Takeru Miyato, Shin-ichi Maeda, Masanori Koyama, and Shin Ishii. Virtual adversarial training: a regularization method for supervised and semi-supervised learning. *IEEE TPAMI*, 41(8):1979–1993, 2018.
- [46] Hang Zhang, Kristin Dana, Jianping Shi, Zhongyue Zhang, Xiaogang Wang, Amrbrish Tyagi, and Amit Agrawal. Context encoding for semantic segmentation. In *CVPR*, pages 7151–7160, 2018.

A Experimental Results

A.1 Hyperparameter setting

For reproduction, we show the detailed hyperparameter setting for each method in Table 3 and 4, for algorithm-dependent and algorithm-independent hyperparameters, respectively.

Table 3: Algorithm dependent parameters.

Algorithm	PL (Flex-PL)	UDA (Flex-UDA)	FixMatch (FlexMatch)
Unlabeled Data to Labeled Data Ratio (CIFAR-10/100, STL-10, SVHN)	1	7	7
Unlabeled Data to Labeled Data Ratio (ImageNet)	-	-	1
Pre-defined Threshold (CIFAR-10/100, STL-10, SVHN)	0.95	0.8	0.95
Pre-defined Threshold (ImageNet)	-	-	0.7
Temperature	-	0.5	-

Table 4: Algorithm independent parameters.

Dataset	CIFAR-10	CIFAR-100	STL-10	SVHN	ImageNet
Model	WRN-28-2 [34]	WRN-28-8	WRN-37-2 [35]	WRN-28-2	ResNet-50 [33]
Weight Decay	5e-4	1e-3	5e-4	5e-4	3e-4
Batch Size	64				128
Learning Rate	0.03				
SGD Momentum	0.9				
EMA Momentum	0.999				
Unsupervised Loss Weight	1				

A.2 Class-wise accuracy improvement.

As introduced in the paper, CPL has its ability of improving performance on those hard-to-learn classes by taking into consider the model’s learning status. A detailed class-wise accuracy comparison is listed in Table 5, where the final accuracies of class 2, 3 and 5 with originally bad performance are improved.

Table 5: Class-wise accuracy comparison on CIFAR-10 40-label split.

Class Number	0	1	2	3	4	5	6	7	8	9
FixMatch	0.964	0.982	0.697	0.852	0.974	0.890	0.987	0.970	0.982	0.981
FlexMatch	0.967	0.980	0.921	0.866	0.957	0.883	0.988	0.975	0.982	0.968

A.3 Median error rates

We also report the median error rates of the last 20 checkpoints by allowing all methods to run the same iterations, following existing work [14]. There are 1000 iterations between every two checkpoints. The results in Table 6 show that our CPL method can dramatically improve the performance of existing SSL algorithms and the FlexMatch achieves the best accuracy. These conclusions are in consistency with the results of Table 1 in the main text, showing the effectiveness of our proposed CPL algorithm.

Table 6: Median error rates of the last 20 checkpoints.

Dataset	CIFAR-10			CIFAR-100			STL-10			SVHN	
Label Amount	40	250	4000	400	2500	10000	40	250	1000	40	1000
PL	77.42 $\pm$ 1.19	48.33 $\pm$ 2.43	15.64 $\pm$ 0.29	90.01 $\pm$ 0.21	58.38 $\pm$ 0.42	37.64 $\pm$ 0.16	76.44 $\pm$ 0.67	56.90 $\pm$ 2.32	33.57 $\pm$ 0.40	69.05 $\pm$ 6.77	9.99 $\pm$ 0.35
Flex-PL	76.09 $\pm$ 2.25	47.53 $\pm$ 2.25	15.30 $\pm$ 0.24	86.60 $\pm$ 0.48	56.72 $\pm$ 0.54	36.20 $\pm$ 0.20	76.84 $\pm$ 1.04	53.71 $\pm$ 2.69	33.19 $\pm$ 0.25	67.20 $\pm$ 3.99	15.10 $\pm$ 1.33
UDA	10.96 $\pm$ 3.68	5.46 $\pm$ 0.07	4.60 $\pm$ 0.05	51.97 $\pm$ 1.38	29.92 $\pm$ 0.35	23.64 $\pm$ 0.33	41.11 $\pm$ 5.21	10.74 $\pm$ 1.39	8.00 $\pm$ 0.58	5.31 $\pm$ 4.39	1.97 $\pm$ 0.04
Flex-UDA	5.77 $\pm$ 0.52	5.48 $\pm$ 0.33	4.52 $\pm$ 0.07	59.51 $\pm$ 2.70	29.33 $\pm$ 0.23	23.38 $\pm$ 0.19	61.16 $\pm$ 4.34	10.88 $\pm$ 0.54	7.16 $\pm$ 0.20	6.21 $\pm$ 2.84	2.13 $\pm$ 0.09
FixMatch	7.99 $\pm$ 0.59	5.12 $\pm$ 0.33	4.46 $\pm$ 0.11	48.95 $\pm$ 1.19	29.19 $\pm$ 0.25	23.06 $\pm$ 0.12	44.70 $\pm$ 6.58	12.34 $\pm$ 2.13	7.38 $\pm$ 0.26	3.92 $\pm$ 1.18	2.06 $\pm$ 0.01
FlexMatch	5.19 $\pm$ 0.05	5.33 $\pm$ 0.12	4.47 $\pm$ 0.09	45.91 $\pm$ 1.76	28.11 $\pm$ 0.20	23.04 $\pm$ 0.28	44.69 $\pm$ 7.49	9.27 $\pm$ 0.49	6.15 $\pm$ 0.25	20.81 $\pm$ 5.26	12.90 $\pm$ 2.68

A.4 Detailed results

To comprehensively evaluate the performance of all methods in a classification setting, we further report the precision, recall, f1 score and AUC (area under curve) results on CIFAR-10 dataset. As shown in Table 7, we see that in addition to the reduced error rates, CPL also has the best performance on precision, recall, F1 score, and AUC. These metrics, together with error rates (accuracy), shows the strong performance of our proposed method.

Table 7: Precision, recall, f1 score and AUC results on CIFAR-10.

Label Amount	40 labels				4000 labels			
Criteria	Precision	Recall	F1 Score	AUC	Precision	Recall	F1 score	AUC
PL	0.2539	0.2552	0.2493	0.6542	0.8498	0.8509	0.8500	0.9833
Flex-PL	0.2865	0.2865	0.2663	0.6718	0.8544	0.8545	0.8542	0.9843
UDA	0.8759	0.8408	0.8086	0.9775	0.9557	0.9559	0.9557	0.9985
Flex-UDA	0.9482	0.9485	0.9482	0.9974	0.9576	0.9577	0.9576	0.9986
Fixmatch	0.9333	0.9290	0.9278	0.9910	0.9571	0.9571	0.9569	0.9984
Flexmatch	0.9506	0.9507	0.9506	0.9975	0.9580	0.9581	0.9580	0.9984

B TorchSSL: A PyTorch-based SSL Codebase

The PyTorch [28] framework has gained increasing attention in the deep learning research community. However, the main existing SSL codebase [43] is based on TensorFlow. For the convenience and customizability, we re-implement and open source a PyTorch-based SSL toolbox, named *TorchSSL*³ as shown in Figure 5. TorchSSL contains eight popular semi-supervised learning methods: II-Model [9], Pseudo-Labeling [4], VAT [45], Mean Teacher [10], MixMatch [12], ReMixMatch [13], UDA [11], and FixMatch [14], along with our proposed method FlexMatch. Most of our implementation details are based on [43]. More importantly, in addition to the basic SSL methods and components, we implement several techniques to make the results stable under PyTorch framework. For instance, we add synchronized batch normalization [46] to avoid the performance degradation caused by multi-GPU training with small batch size, and a batch norm controller to prevent performance crashes for some algorithms, which is not officially supported in PyTorch.

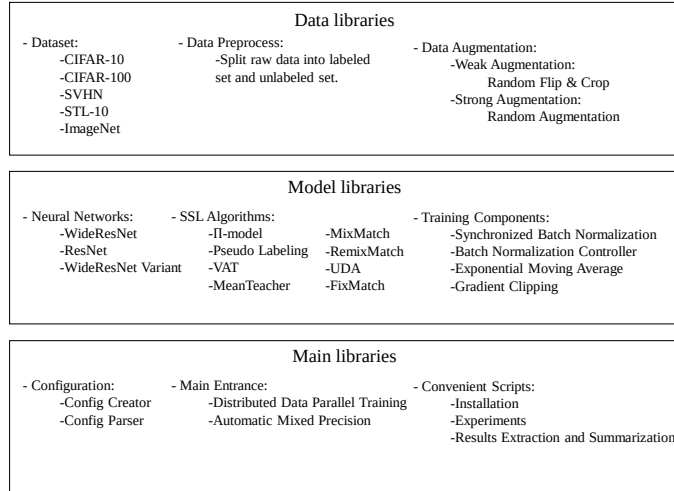


Figure 5: Components of TorchSSL.

³Our toolbox is partially based on [44].

B.1 BatchNorm Controller

We observed that Mean Teacher can be very unstable if we update BatchNorm for both labeled data and unlabeled data in turn. Other algorithms such as Π -Model and MixMatch also show the similar instability. Therefore, we use BatchNorm Controller to update BatchNorm only for labeled data if labeled data and unlabeled data are forwarded separately. The code of BatchNorm Controller is as follows. We record the BatchNorm statistics before the forward propagation of unlabeled data and restore them after the propagation is done.

B.2 Benchmark results

We comprehensively run all algorithms in our TorchSSL on four common datasets in SSL: CIFAR-10, CIFAR-100, SVHN, and STL-10, and report the best error rates in Table 8, 9, 10, and 11, respectively. These benchmark results provide a reference of using this toolbox.

Table 8: Benchmark results on CIFAR-10. The error bars are obtained from three trials.

Algorithms	Error Rate (40 labels)	Error Rate (250 labels)	Error Rate (4000 labels)
Π -Model [9]	74.34 $\pm$ 1.76	46.24 $\pm$ 1.29	13.13 $\pm$ 0.59
Pseudo-Labeling [4]	74.61 $\pm$ 0.26	46.49 $\pm$ 2.20	15.08 $\pm$ 0.19
VAT [45]	74.66 $\pm$ 2.12	41.03 $\pm$ 1.79	10.51 $\pm$ 0.12
Mean Teacher [10]	70.09 $\pm$ 1.60	37.46 $\pm$ 3.30	8.10 $\pm$ 0.21
MixMatch [12]	36.19 $\pm$ 6.48	13.63 $\pm$ 0.59	6.66 $\pm$ 0.26
ReMixMatch [13]	9.88 $\pm$ 1.03	6.30 $\pm$ 0.05	4.84 $\pm$ 0.01
UDA [11]	10.62 $\pm$ 3.75	5.16 $\pm$ 0.06	4.29 $\pm$ 0.07
FixMatch [14]	7.47 $\pm$ 0.28	4.86 $\pm$ 0.05	4.21 $\pm$ 0.08
FlexMatch	4.97 $\pm$ 0.06	4.98 $\pm$ 0.09	4.19 $\pm$ 0.01

Table 9: Benchmark results on CIFAR-100.

Algorithms	Error Rate (400 labels)	Error Rate (2500 labels)	Error Rate (10000 labels)
Π -Model [9]	86.96 $\pm$ 0.80	58.80 $\pm$ 0.66	36.65 $\pm$ 0.00
Pseudo-Labeling [4]	87.45 $\pm$ 0.85	57.74 $\pm$ 0.28	36.55 $\pm$ 0.24
VAT [45]	85.20 $\pm$ 1.40	46.84 $\pm$ 0.79	32.14 $\pm$ 0.19
Mean Teacher[10]	81.11 $\pm$ 1.44	45.17 $\pm$ 1.06	31.75 $\pm$ 0.23
MixMatch [12]	67.59 $\pm$ 0.66	39.76 $\pm$ 0.48	27.78 $\pm$ 0.29
ReMixMatch [13]	42.75 $\pm$ 1.05	26.03 $\pm$ 0.35	20.02 $\pm$ 0.27
UDA [11]	46.39 $\pm$ 1.59	27.73 $\pm$ 0.21	22.49 $\pm$ 0.23
FixMatch [14]	46.42 $\pm$ 0.82	28.03 $\pm$ 0.16	22.20 $\pm$ 0.12
FlexMatch	39.94 $\pm$ 1.62	26.49 $\pm$ 0.20	21.90 $\pm$ 0.15

Table 10: Benchmark results on STL-10.

Algorithms	Error Rate (40 labels)	Error Rate (250 labels)	Error Rate (1000 labels)
Π -Model [9]	74.31 $\pm$ 0.85	55.13 $\pm$ 1.50	32.78 $\pm$ 0.40
Pseudo-Labeling [4]	74.68 $\pm$ 0.99	55.45 $\pm$ 2.43	32.64 $\pm$ 0.71
VAT [45]	74.74 $\pm$ 0.38	56.42 $\pm$ 1.97	37.95 $\pm$ 1.12
Mean Teacher [10]	71.72 $\pm$ 1.45	56.49 $\pm$ 2.75	33.90 $\pm$ 1.37
MixMatch [12]	54.93 $\pm$ 0.96	34.52 $\pm$ 0.32	21.70 $\pm$ 0.68
ReMixMatch [13]	32.12 $\pm$ 6.24	12.49 $\pm$ 1.28	6.74 $\pm$ 0.14
UDA [11]	37.42 $\pm$ 8.44	9.72 $\pm$ 1.15	6.64 $\pm$ 0.17
FixMatch [14]	35.97 $\pm$ 4.14	9.81 $\pm$ 1.04	6.25 $\pm$ 0.33
FlexMatch	29.15 $\pm$ 4.16	8.23 $\pm$ 0.39	5.77 $\pm$ 0.18

Table 11: Benchmark results on SVHN.

Algorithms	Error Rate (40 labels)	Error Rate (250 labels)	Error Rate (1000 labels)
Π -Model [9]	67.48 ± 0.95	13.30 ± 1.12	7.16 ± 0.11
Pseudo-Labeling [4]	64.61 ± 5.60	15.59 ± 0.95	9.40 ± 0.32
VAT [45]	74.75 ± 3.38	4.33 ± 0.12	4.11 ± 0.20
Mean Teacher [10]	36.09 ± 3.98	3.45 ± 0.03	3.27 ± 0.05
MixMatch [12]	30.60 ± 8.39	4.56 ± 0.32	3.69 ± 0.37
ReMixMatch [13]	24.04 ± 9.13	6.36 ± 0.22	5.16 ± 0.31
UDA [11]	5.12 ± 4.27	1.92 ± 0.05	1.89 ± 0.01
FixMatch [14]	3.81 ± 1.18	2.02 ± 0.02	1.96 ± 0.03
FlexMatch	8.19 ± 3.20	6.59 ± 2.29	6.72 ± 0.30